

 shieldify

SMART CONTRACT SECURITY ASSESSMENT

Pear

— VAULT

SECURITY REVIEW

5 SEPTEMBER 2026

rt contracts, our shielding · Your smart contracts, our shielding · Your smart contracts

Contents

1	About Shieldify	3
2	Disclaimer	3
3	About Pear	3
4	Risk Classification	4
4.1	Impact	4
4.2	Likelihood	4
5	Security Review Summary	4
5.1	Protocol Summary	5
5.2	Scope	5
6	Findings Summary	5
7	Findings	6

1 About Shieldify

Positioned as the first hybrid Web3 Security company, Shieldify shakes things up with a unique subscription-based auditing model that entitles the customer to unlimited audits within its duration, as well as top-notch service quality thanks to a disruptive 6-layered security approach. The company works with very well-established researchers in the space and has secured multiple millions in TVL across protocols, also can audit codebases written in Solidity, Vyper, Rust, Cairo, Move and Go.

Learn more about us at shieldify.org.

2 Disclaimer

This security review does not guarantee bulletproof protection against a hack or exploit. Smart contracts are a novel technological feat with many known and unknown risks. The protocol, which this report is intended for, indemnifies Shieldify Security against any responsibility for any misbehavior, bugs, or exploits affecting the audited code during any part of the project's life cycle. It is also pivotal to acknowledge that modifications made to the audited code, including fixes for the issues described in this report, may introduce new problems and necessitate additional auditing.

3 About Pear

Pear is a system of ERC-4626 vaults on HyperEVM where investors pool capital and a designated leader runs delegated pair-trading on Hyperliquid, without ever taking custody of investor funds.

Capital lives on-chain in two places: the vault contract itself on the EVM side, and a Privy-custodied agent wallet that holds the trading position. Trade execution happens off-chain - `pear-vault-service` handles signing, and the `pear-pro` / `pear-hl-engine` trading APIs route the orders. The leader can therefore direct strategy without ever being able to move investor principal out of the system.

At a high level, the main functionalities are:

- Investors deposit an asset token into a vault and receive ERC-4626 shares
- A leader directs delegated pair-trading on Hyperliquid through an agent wallet bound to the vault
- Share price is derived from a composite net-asset-value that spans the vault's EVM balance, the agent's wallet balance, the agent's HyperCore perp and spot positions, a Felix vault position, and assets in flight across the EVM-HyperCore bridge
- Exits run through two paths: an immediate direct withdrawal when the vault holds enough spare liquidity, or a withdrawal queue settled later by an admin
- A Comptroller holds protocol-wide configuration - vault configs, fee parameters, and per-user deposit caps derived from PEAR tiers
- A factory deploys and registers vaults, each bound to its own agent

4 Risk Classification

SEVERITY	IMPACT: HIGH	IMPACT: MEDIUM	IMPACT: LOW
Likelihood: High	 CRITICAL	 HIGH	 MEDIUM
Likelihood: Medium	 HIGH	 MEDIUM	 LOW
Likelihood: Low	 MEDIUM	 LOW	 LOW

4.1 Impact

- **High** - results in a significant risk for the protocol's overall well-being. Affects all or most users
- **Medium** - results in a non-critical risk for the protocol affects all or only a subset of users, but is still unacceptable
- **Low** - losses will be limited but bearable, and covers vectors similar to griefing attacks that can be easily repaired

4.2 Likelihood

- **High** - almost certain to happen and highly lucrative for execution by malicious actors
- **Medium** - still relatively likely, although only conditionally possible
- **Low** - requires a unique set of circumstances and poses non-lucrative cost-of-execution to rewards ratio for the actor

5 Security Review Summary

This is an extended review of the Pear vault system, following on from an earlier engagement on the same codebase.

The security review lasted 6 days with a total of 96 hours dedicated to the audit by the Shieldify team.

Overall, the code is well-written. The audit report contributed by identifying two Medium and four Low issues. They're mainly related to asset accounting, withdrawal handling, fee calculations and share-price accuracy.

The Pear team has done a great job with their test suite and provided support and responses to all of the questions that the Shieldify researchers had.

5.1 Protocol Summary

PROJECT NAME	PEAR
Repository	pear-vault-smartcontracts
Type of Project	ERC-4626 Delegated Trading Vaults
Security Review Timeline	6 days
Review Commit Hash	40a7a7cb02dce17c1a35189f8b2f4c5c0c0cec8d
Fixes Review Commit Hash	d39c7ff8f78edc77bc67a585c7653cd67baf50a7

5.2 Scope

The changes in the following smart contracts were in the scope of the security review:

FILE	NSLOC
src/PearVaultFactory.sol	116
src/libraries/HyperliquidHelper.sol	66
src/Comptroller.sol	249
src/PearVault.sol	744
src/hyperliquid/L1Read.sol	264
src/hyperliquid/ICoreWriter.sol	3
src/CoreWriterReadCaller.sol	86
Total	1528

6 Findings Summary

The following number of issues have been identified, sorted by their severity:

- **Medium** issues: 2
- **Low** issues: 4
- **Info** issues: 5

ID	TITLE	SEVERITY	STATUS
[M-01]	Direct Withdrawals Spend The Assets Set Aside For Queued Withdrawals	MEDIUM	FIXED
[M-02]	Money Held In Other HyperCore Spot Tokens Is Missing From The Share Price	MEDIUM	FIXED
[L-01]	The Performance Fee Makes <code>previewRedeem()</code> Disagree With What You Actually Get	LOW	FIXED
[L-02]	The Minimum Withdrawal Fee Underflows On Small Requests And Blocks Whole Batches	LOW	FIXED
[L-03]	<code>totalAssets()</code> Reverts While Money Moves Back From HyperCore, Freezing Every Exit	LOW	FIXED
[L-04]	A Vault Owner Can Keep Deposits That Mint Zero Shares, And Can Repeat It	LOW	FIXED
[I-01]	Two Vaults Sharing One Agent Count The Same Money Twice	INFO	FIXED
[I-02]	Nothing Checks That A Config's Asset And System Address Use The Same Unit	INFO	FIXED
[I-03]	A Second Withdrawal Request Subtracts Locked Shares Twice	INFO	FIXED
[I-04]	<code>maxDeposit()</code> And <code>maxMint()</code> Ignore The Per User Cap	INFO	FIXED
[I-05]	<code>_minCashFlowBPS()</code> Mislabeled Live Vault Balance as Total Deposits Percentage	INFO	FIXED

7 Findings

M-01

MEDIUM

FIXED

Direct Withdrawals Spend The Assets Set Aside For Queued Withdrawals

Description

The vault keeps two exit paths. A user can withdraw straight away if the vault holds enough spare tokens, or the user joins a queue and an admin pays them later.

`_canDirectWithdraw()` is the guard that keeps these two paths apart. It is supposed to let a direct withdrawal through only when the vault can pay that user and still cover everyone already waiting in the queue.

The guard compares the wrong number. It checks the vault balance against `amount`, which is the net figure the user asked for. The withdrawal then pays out `amount` plus the withdrawal fee plus any performance fee, because fees are taken out of the vault balance too and sent to the fee recipient.

So the vault spends more than the guard measured. The gap is exactly `withdrawalFee + performanceFee`. That money comes out of the reserve meant for queued users, and a queued withdrawal that was fully funded a moment ago can no longer be paid.

The gap is bounded. `MAX_WITHDRAWAL_FEE_BPS` is 1000 and `MAX_PERFORMANCE_FEE_BPS` is 3000, so a single withdrawal can overspend by at most 40% of its own size. Repeated withdrawals repeat the overspend.

Location of Affected Code

File: [src/PearVault.sol#L1089-L1103](#)

```
function _canDirectWithdraw(uint256 amount) internal view returns (bool) {
    uint256 vaultBalance = vaultToken().balanceOf(address(this));

    // Calculate total assets needed for all pending withdrawal requests
    uint256 pendingWithdrawalAssets = 0;
    if (totalPendingShares > 0) {
        // Convert pending shares to assets (this is an estimate as share price may change)
        pendingWithdrawalAssets = super.previewRedeem(totalPendingShares);
    }

    // Direct withdrawal is allowed only if vault has enough to cover:
    // 1. The current withdrawal request (amount)
    // 2. All pending queued withdrawals (pendingWithdrawalAssets)
    return vaultBalance >= amount + pendingWithdrawalAssets;
}
```

File: [src/PearVault.sol#L784-L803](#)

```
function withdraw(uint256 assets, address receiver, address owner) public override(ERC4626Upgradeable, IERC4626)
nonReentrant onlyActiveVault returns (uint256) {
    if (!_canDirectWithdraw(assets)) {
        revert DirectWithdrawNotAllowed();
    }

    // Use our fee-adjusted previewWithdraw to get correct shares needed
    uint256 shares = previewWithdraw(assets);
    // code
    _withdrawWithFee(shares, owner, receiver);
    return shares;
}
```

File: [src/PearVault.sol#L1315-L1328](#)

```

function withdraw(uint256 assets, address receiver, address owner) public override(ERC4626Upgradeable, IERC4626)
nonReentrant onlyActiveVault returns (uint256) {
    // code
    // Settle fees (withdrawal + performance on realized profit) and update cost basis
    (uint256 assetsToTransfer, uint256 withdrawalFee, uint256 performanceFee) =
        _settleWithdrawalFees(user, assetsBeforeFee, shares, userTotalShares);

    // Use ERC4626's withdraw logic: burns shares + transfers assets + emits event
    super._withdraw(
        user,      // caller (same as owner to avoid allowance check)
        receiver,  // receiver
        user,      // owner of shares
        assetsToTransfer, // assets to transfer (after fees)
        shares     // shares to burn
    );

    // Transfer fees
    _transferFees(user, withdrawalFee, performanceFee);
    // code
}

```

Impact

A queued withdrawal that the contract had already reserved money for becomes unpayable. The queued user is not warned, and the failure only shows up when the admin tries to settle the queue and the call reverts.

Any share holder can cause this. No special role is needed and no coordination is needed. It is not a permanent loss, because an admin can bridge money back from HyperCore and settle the queue afterwards, but until that happens the queue is stuck and the contract has broken a rule it states in its own comments.

Proof of Concept

The vault is left holding exactly `queuedGross + requestedNet`, which is the amount the guard treats as sufficient. One direct withdrawal then makes the queued withdrawal unpayable.

```

function test_DirectWithdrawConsumesLiquidityReservedForQueuedShares() public {
    address user2 = makeAddr("user2");
    token.mint(user2, DEPOSIT);

    _deposit(user1, DEPOSIT);

    // user2 queues a withdrawal for their whole position.
    vm.startPrank(user2);
    token.approve(address(vault), DEPOSIT);
    uint256 queuedShares = vault.deposit(DEPOSIT, user2);
    vault.requestWithdrawal(queuedShares, user2);
}

```

```

vm.stopPrank();

uint256 queuedGross = vault.convertToAssets(queuedShares);
uint256 requestedNet = 9_000e6;
uint256 targetEvmBalance = queuedGross + requestedNet;
uint256 evmBalance = token.balanceOf(address(vault));

// Normal strategy placement: funds are visible in the agent's HyperCore spot
// balance but are not immediately available as EVM ERC20 liquidity.
uint256 strategyAssets = evmBalance - targetEvmBalance;
vm.prank(address(vault));
token.transfer(lossSink, strategyAssets);
vm.mockCall(
    SPOT_BALANCE_PRECOMPILE,
    abi.encode(agent, uint64(291)),
    abi.encode(uint64(strategyAssets * 100), uint64(0), uint64(0))
);

// Preserve the canonical max-allowance setup. Execution still fails because
// the assets are on HyperCore, not in the agent's EVM token balance.
vm.prank(agent);
token.approve(address(vault), type(uint256).max);

// 1. The guard approves the withdrawal.
assertTrue(vault.canDirectWithdraw(requestedNet));

// 2. The withdrawal goes through.
vm.prank(user1);
vault.withdraw(requestedNet, user1, user1);

// 3. The vault now holds less than the queued withdrawal needs. It paid out
// requestedNet plus withdrawalFee plus performanceFee, but the guard
// only ever measured requestedNet.
assertLt(token.balanceOf(address(vault)), queuedGross);

// 4. Settling the queued withdrawal reverts.
vm.prank(pearAdmin);
vm.expectRevert();
vault.executeWithdrawal(1);
}

```

Recommendation

Measure the same amount that the withdrawal will actually spend. Work out the gross amount first, then check it:

```

uint256 shares = previewWithdraw(assets);
uint256 grossAssets = super.previewRedeem(shares);
if (!_canDirectWithdraw(grossAssets)) revert DirectWithdrawNotAllowed();

```

Do the same in `redeem()`, which shares the guard. A cheap regression test is to fund a vault with exactly `queuedGross + requestedNet`, run a direct withdrawal, and assert that settling the queue still works.

Team Response

Fixed.

M-02

MEDIUM

FIXED

Money Held In Other HyperCore Spot Tokens Is Missing From The Share Price

Description

`totalAssets()` is what sets the share price. It adds up six things: the vault's own token balance, the agent's wallet balance, the agent's perp account value, the agent's spot balance, any Felix vault position, and money in flight across the bridge.

The spot part only looks at two token IDs. It reads `vaultConfig.assetTokenId`, and then it reads token id `0` as well. Nothing else is counted.

The agent trades on HyperCore and can hold any spot token. The moment the agent holds value in a third token, that value is invisible to the vault. The share price drops even though the vault did not lose anything.

This is wrong on its own, with nobody attacking. Anyone depositing while the agent holds an uncounted token pays too little for their shares, and anyone redeeming at that moment gets too little back.

It is also something a vault owner can steer. The owner controls where the strategy sits. Move value into an uncounted token and the price drops. Deposit at the low price. Move value back and the price returns. Redeem. The extra shares now claim real money that belonged to the people already in the vault.

The per-user deposit cap does not slow this down. `deposit()` skips the cap check when `receiver == owner()`, so the owner's buy at the depressed price is limited only by how much money they have.

Location of Affected Code

File: [src/PearVault.sol#L444-L500](#)

```
function totalAssets() public view override (ERC4626Upgradeable, IERC4626) returns (uint256) {
    // 1. Vault asset token balance (6 decimals)
    uint256 vaultAssetBalance = vaultToken().balanceOf(address(this));

    // 2. Agent wallet balance (6 decimals)
```

```

uint256 agentWalletBalance = IERC20(vaultConfig.assetToken).balanceOf(agent);

//code

// 4. Agent asset token spot balance (8→6 decimals)
uint256 agentAssetSpot = HyperliquidHelper.getAgentSpotBalance(
    agent,
    vaultConfig.assetTokenId
) / 100;

// Agent Spot USD Balance (0 token id ) and add to agentAssetSpot
if(vaultConfig.assetTokenId != 0){
    agentAssetSpot += HyperliquidHelper.getAgentSpotBalance(
        agent,
        0
    ) / 100;
}
//code
}

```

File: [src/PearVault.sol#L651-L656](#)

```

function deposit(uint256 assets, address receiver) public override(ERC4626Upgradeable, IERC4626) nonReentrant
onlyActiveVault returns (uint256) {
    // code
    if (receiver != owner()) {
        uint256 maxAllowed = comptroller().getMaxAllowedInvestment(receiver);
        if (userDepositAmount[receiver] + assets > maxAllowed) {
            revert DepositLimitExceeded();
        }
    }
    // code
}

```

Impact

Two separate problems come out of the same line.

- The everyday one: deposits and redemptions are mispriced whenever the agent holds a token the vault does not read. Nobody has to be attacking. Depositors gain at the expense of holders, or lose, depending on which way the position sits.
- The steerable one: a vault owner can move the price down, buy in, move it back, and sell. The proof below shows the owner ending up ahead and an existing holder ending up behind, in one round trip. The owner never takes custody of anyone's money and never breaks the agent's withdrawal rules, so nothing else in the system flags it.

The profit comes from the missing token ID and not from the setup. Running the identical round trip against a token the vault *does* count leaves the owner down 1,175,284,195 raw units on fees, rather than up. That is the check that rules out a fee or rounding artefact.

How much of this is reachable depends on which spot markets the trading engine actually routes to. We reviewed the contracts and the merged docs, not the deployed engine config. If the engine can route to markets outside the two token IDs, this deserves a higher rating than Medium. That is the one link in this finding we could not check ourselves, and it is worth confirming with the engine team.

Proof of Concept

The owner moves value into a spot token the vault does not read, deposits at the depressed price, moves it back, and exits. Moving value is modelled by lowering the agent's balance in the configured token ID, since `totalAssets()` reads only that ID and token ID `0`.

```
function test_LeaderExtractsValueViaUnenumeratedSpotRoundTrip() public {
    uint256 victimShares = _deposit(user1, DEPOSIT);
    uint256 strategyAssets = 50_000e6;

    // Strategy value sits in the configured spot token, so NAV counts it.
    vm.mockCall(
        SPOT_BALANCE_PRECOMPILE,
        abi.encode(agent, uint64(291)),
        abi.encode(uint64(strategyAssets * 100), uint64(0), uint64(0))
    );

    uint256 victimValueBefore = vault.convertToAssets(victimShares);
    uint256 leaderTokensBefore = token.balanceOf(owner);

    // 1. Leader routes a spot swap into an unenumerated token. NAV drops by the
    // full amount, because no other token id is read.
    vm.mockCall(
        SPOT_BALANCE_PRECOMPILE,
        abi.encode(agent, uint64(291)),
        abi.encode(uint64(0), uint64(0), uint64(0))
    );
    uint256 depressedNav = vault.totalAssets();

    // 2. Leader deposits at the depressed price. No tier cap applies to owner.
    uint256 attackDeposit = 100_000e6;
    vm.startPrank(owner);
    token.approve(address(vault), attackDeposit);
    uint256 leaderShares = vault.deposit(attackDeposit, owner);
    vm.stopPrank();

    // 3. Leader swaps back. NAV restores; the extra shares now claim real assets.
    vm.mockCall(
        SPOT_BALANCE_PRECOMPILE,
        abi.encode(agent, uint64(291)),
```

```

    abi.encode(uint64(strategyAssets * 100), uint64[0], uint64[0])
  };

  // 4. Leader exits.
  vm.prank(owner);
  vault.redeem(leaderShares, owner, owner);

  uint256 victimValueAfter = vault.convertToAssets(victimShares);
  int256 leaderPnl = int256(token.balanceOf(owner)) - int256(leaderTokensBefore);

  emit log_named_uint("depressedNav", depressedNav);
  emit log_named_uint("leaderDeposited", attackDeposit);
  emit log_named_int("leaderNetPnl", leaderPnl);
  emit log_named_uint("victimValueBefore", victimValueBefore);
  emit log_named_uint("victimValueAfter", victimValueAfter);

  assertGt(leaderPnl, int256(0), "leader profits on the round trip");
  assertLt(victimValueAfter, victimValueBefore, "existing holder loses value");
}

```

Observed result

```

depressedNav: 110000000000
leaderDeposited: 100000000000
leaderNetPnl: 22345591702
victimValueBefore: 14545454545
victimValueAfter: 12380952380

```

NAV falls by the full 50,000e6 the moment the value moves, which is the omission itself. The owner then puts in 100,000e6 and comes out 22,345e6 ahead, while the existing holder's stake falls from 14,545e6 to 12,380e6.

Recommendation

Stop hardcoding the token list. Options, roughly in order of how much work they are:

1. Keep a list of spot token IDs in `VaultConfig` and loop over it in `totalAssets()`. Let the Comptroller owner add IDs as new markets are enabled.
2. Restrict the trading engine so the agent can only ever hold the two IDs the vault reads, and assert that somewhere the vault can see.
3. Have the agent report its own spot inventory, and treat anything it cannot account for as zero rather than silently dropping it.

Whichever you pick, add a test that puts value in a token the vault does not know about and asserts the share price does not move. That is the property that broke here.

Until this is fixed, the owner's exemption from the deposit cap makes the profitable version larger than it needs to be. Consider applying the cap to the owner too, or at least capping owner deposits made while the share price is below its recent range.

Team Response

Fixed.

L-01

• LOW

FIXED

The Performance Fee Makes `previewRedeem()` Disagree With What You Actually Get

Description

This bug was not introduced by a fix. A later feature reopened a rule an earlier fix had closed. The previous review reported L-09 part one, `withdraw()` not handing back the amount asked for. That was taken care of and is still correct. What reopened the standard is the performance fee, added in commit `49bdd50` as a feature rather than a remediation, months after that review. The fee is charged at withdrawal but never included in the preview, so the same guarantee is broken again in a different place.

ERC-4626 says `previewRedeem()` must return the exact number of assets a redemption will produce, and that it must not return more than the real call gives back.

`previewRedeem()` here subtracts the withdrawal fee. It does not subtract the performance fee. The performance fee is worked out at withdrawal time from the user's cost basis, and the preview function has no access to that number.

So the preview is too high for any user sitting on a profit. Integrators read the preview and expect that many assets get less.

`previewWithdraw()` has the same gap on the other side, and `maxWithdraw` is derived from the same numbers.

Location of Affected Code

File: [src/PearVault.sol#L882-L887](#)

```
function previewRedeem(uint256 shares) public view override(ERC4626Upgradeable, IERC4626) returns (uint256) {
    uint256 assets = super.previewRedeem(shares);
    return assets - _feeOnTotal(assets, _exitFeeBasisPoints());
}
```

File: [src/PearVault.sol#L890-L893](#)

```
function previewWithdraw(uint256 assets) public view override(ERC4626Upgradeable, IERC4626) returns (uint256) {
    uint256 fee = _feeOnRaw(assets, _exitFeeBasisPoints());
    return super.previewWithdraw(assets + fee);
}
```

The performance fee is applied later, in `_settleWithdrawalFees()`, called from `src/PearVault.sol:1315`. Nothing in the preview path reaches it.

Impact

This is an interface problem, not a theft. Nobody loses money that was theirs. What breaks is any integrator that trusts the preview, which is exactly what the standard tells them to do. Automated strategies that size a redemption from `previewRedeem()` will come up short, and a contract that reverts on a shortfall will revert.

The size of the gap is the performance fee on realised profit, capped at 30% of profit.

Recommendation

Work out the performance fee inside the preview. The inputs are all readable on-chain:

`userCostBasis[owner]`, the user's total shares, and `_performanceFeeBPS()`. Pull the fee maths into a shared internal function and call it from both the preview and the settlement path so the two cannot drift apart again.

Because the fee depends on who owns the shares, the strictly correct preview needs an owner argument. ERC-4626 does not give you one. Two honest ways out:

- Add `previewRedeemFor(uint256 shares, address owner)` and say clearly in the docs that plain `previewRedeem()` gives the no fee upper bound.
- Have `previewRedeem()` assume the worst case, a full performance fee on the whole amount, so it can never promise more than the user gets.

The second keeps you inside the standard's rule that the preview must not overstate.

Team Response

Fixed.

L-02

LOW

FIXED

The Minimum Withdrawal Fee Underflows On Small Requests And Blocks Whole Batches

Description

This bug was introduced by the fix for an earlier finding. The previous review reported L-03, tiny withdrawals rounding their fee down to zero so users could split a withdrawal and pay nothing. That was taken care of: commit `ed810c3` added a minimum fee floor and fee avoidance is closed. What was not taken care of is the other end of the range. The floor is now applied without checking it against the amount being withdrawn, so small withdrawals owe more than they are worth. Before the fix there was no floor and nothing could underflow. The fix inverted the bug rather than removing it.

Every withdrawal charges at least a floor fee. `_calculateWithdrawalFee()` takes the percentage fee and raises it to `getMinFeeAmount()` if the percentage came out lower.

Nothing checks that the floor is smaller than the amount being withdrawn. When a user withdraws a very small amount, the fee is larger than the amount, and the subtraction that takes the fee off the payout underflows and reverts.

This is reachable with the values the contract ships with. `Comptroller.initialize()` sets `_minFeeAmount = 1000`. `requestWithdrawal()` only rejects zero, so a one share request goes straight in. One share is worth about 1 raw unit, and the floor fee is 1000.

The queue makes it worse than a single failed call. `batchExecuteWithdrawal()` settles a list of requests in one transaction. One dust request in the list reverts the whole batch, so other users' withdrawals do not get paid either.

Location of Affected Code

File: [src/PearVault.sol#L1427-L1431](#)

```
function _calculateWithdrawalFee(uint256 assetsBeforeFee) internal view returns (uint256) {
    uint256 calculatedFee = _feeOnTotal(assetsBeforeFee, _exitFeeBasisPoints());
    uint256 minFee = comptroller().getMinFeeAmount();
    return calculatedFee < minFee ? minFee : calculatedFee;
}
```

File: [src/Comptroller.sol#L106](#)

```
function initialize(address initialOwner, address pearAdmin_) external initializer {
    // code
    _minFeeAmount = 1000;
    // code
}
```

```
function requestWithdrawal(uint256 shares,address receiver) external override nonReentrant validAmount(shares)
```

Impact

- A user with a tiny position cannot get out. Their withdrawal reverts and no amount of retrying helps.
- More importantly, one dust request stalls the shared batch path. Other users waiting in the same batch do not get paid until an admin notices, works out which request is the problem, and either force-cancels it or rebuilds the batch without it.

This is recoverable. `forceCancelWithdrawal()` exists and an admin can settle requests one at a time. That is why it is Low and not Medium. But it is unprivileged, it costs the attacker almost nothing, and it can be repeated, so it is a cheap way to make the queue annoying to run.

Recommendation

Cap the fee at the amount being withdrawn:

```
uint256 fee = calculatedFee < minFee ? minFee : calculatedFee;  
return fee > assetsBeforeFee ? assetsBeforeFee : fee;
```

That stops the underflow, but a user whose whole withdrawal is eaten by the fee still gets nothing. Better to stop the request earlier. Reject a `requestWithdrawal()` whose gross value is below the fee floor, so the user finds out at request time rather than at settlement time.

Separately, make `batchExecuteWithdrawal()` skip a failing request instead of reverting the batch. One bad entry should not hold up everyone else. Emit an event for the skipped ID so an operator can follow up.

Team Response

Fixed.

L-03

LOW

FIXED

`totalAssets()` Reverts While Money Moves Back From HyperCore, Freezing Every Exit

Description

This bug was introduced by the fix for an earlier finding. The previous review reported M-05: assets in flight to HyperCore being invisible to `totalAssets()`, so a depositor could mint shares at a deflated price. That was taken care of: commit `7ad72bc` added the whole in-flight mechanism and the share inflation is closed. What was not taken care of is that the two directions were handled differently. The to Core branch adds the amount and carries on. The from Core branch reverts, and that same commit is where `revert InFlightTransferPending()` was introduced. Before it, there was no in-flight code and nothing to revert.

When an admin brings money back from HyperCore to the EVM side, they first call `reportInFlightFromCore()` to record it. Until the money lands, the vault cannot see it in any balance.

The contract handles this by making `_getInFlightAssets()` revert while a transfer from Core is pending. `_getInFlightAssets()` is called by `totalAssets()`, and `totalAssets()` is called by nearly everything.

So during that window, the vault does not just block deposits. It blocks redemptions, it blocks direct withdrawals, it blocks settling the queue, and it makes every ERC-4626 view function revert, including `totalAssets()`, `convertToAssets()`, `previewRedeem()` and `maxWithdraw()`.

The queue is documented as the fallback for when direct withdrawal is not available. During this window, both are shut, so there is no way out at all.

Notice the asymmetry: the to Core direction adds the amount and carries on. Only the from Core direction reverts.

Location of Affected Code

File: [src/PearVault.sol#L178-L193](#)

```
function _getInFlightAssets() internal view returns (uint256) {
    uint64 currentL1Block = HyperliquidHelper.getL1BlockNumber();
    uint256 inFlightAmount = 0;

    // Check EVM → Core in-flight transfer
    if (inFlightToCore.amount > 0 && currentL1Block < inFlightToCore.creditingBlock) {
        inFlightAmount += inFlightToCore.amount;
    }

    // Check Core → EVM in-flight transfer
    if (inFlightFromCore.amount > 0 && currentL1Block < inFlightFromCore.creditingBlock) {
        revert InFlightTransferPending();
    }
}
```

```
}  
  
return inFlightAmount;  
}
```

Called from `totalAssets()` at `src/PearVault.sol:491`, which every pricing and exit path goes through.

Impact

Every exit is closed while the window is open. Users cannot redeem, cannot withdraw, and cannot have a queued withdrawal settled. View functions revert too, so a front end cannot even show a balance and an integrating contract that reads `convertToAssets()` reverts along with it.

Only the Pear admin can open the window, and it closes on its own when the crediting block arrives, so this is not something an outsider can trigger or hold open. That is why it is Low. But the reason it exists at all- that the money is temporarily invisible- is solvable without shutting the vault.

Recommendation

Treat the from Core direction the same way as the to Core direction. Add the reported amount to `totalAssets()` instead of reverting:

```
if (inFlightFromCore.amount > 0 && currentL1Block < inFlightFromCore.creditingBlock) {  
    inFlightAmount += inFlightFromCore.amount;  
}
```

The reason it reverts today is presumably to stop someone minting shares against money that has not arrived. That worry is real, but it belongs on the deposit path, not on the exit path. Block deposits while a transfer from Core is pending and let exits run normally. The people trying to leave are not the risk.

`reportInFlightFromCore()` is already `onlyPearAdmin`, so the reported amount cannot be faked by the vault owner.

Team Response

Fixed.

L-04

LOW

FIXED

A Vault Owner Can Keep Deposits That Mint Zero Shares, And Can Repeat It

Description

This is the classic ERC-4626 first depositor problem, plus a second stage that makes it worse than the classic version.

The first stage is the familiar one. `loadFirstDeposit()` does not require a minimum amount, so the owner can seed the vault with a single raw unit. The owner then sends tokens straight to the vault address. That raises the assets without raising the share count, so the price per share goes up. Push it far enough and a normal-sized deposit rounds down to zero shares. The depositor's money is now in the vault and they own none of it.

The second stage is what makes this different from the textbook case. OpenZeppelin's protection would normally leave the attacker out of pocket, because the value the attacker gains is partly absorbed by the virtual share and stranded. Here it is not stranded. The owner redeems the last remaining share, which takes `totalSupply()` back to zero. At zero supply, the vault's own overrides kick in, the price resets to 1:1, and the leftover assets become claimable again by whoever deposits next. The owner deposits, takes the leftovers, and is ahead.

And because there is no floor on how far `redeem()` can take the supply down, the owner can go back to a one-share supply and do the whole thing again. The donation needed for the second victim is the same as the first.

Why OpenZeppelin's usual protection does not stop this

The vault inherits `ERC4626Upgradeable` from OpenZeppelin 5.4.0, which already carries the standard defence. It is worth being precise about why that defence is not doing its job here, because the usual advice will not fix it.

OpenZeppelin's defence is virtual shares and virtual assets. Conversions run as if the vault holds one extra asset and `10**_decimalsOffset()` extra shares that belong to nobody:

```
// lib/openzeppelin-contracts-upgradeable/./ERC4626Upgradeable.sol:248
return assets.mulDiv(totalSupply() + 10**_decimalsOffset(), totalAssets() + 1, rounding);
```

The point of those phantom units is that they take a cut of any donation. The [OpenZeppelin documentation](#) puts the guarantee this way: "If the offset is 0, the attacker's loss is at least equal to the user's deposit." In other words, the attack is not meant to pay. Raising `_decimalsOffset()` makes it cost even more, which is the [standard recommendation](#).

Three things break that guarantee here.

One: the vault overrides the maths away. `_convertToShares()` short-circuits before OpenZeppelin's formula ever runs when supply is zero:

```
// src/PearVault.sol:609-619
function _convertToShares(
    uint256 assets,
    Math.Rounding rounding
) internal view override returns (uint256) {
    // For initial deposit (totalSupply == 0), return 1:1 ratio to avoid rounding to 0
    if (totalSupply() == 0) {
        return assets;
    }
    // Otherwise use standard ERC4626 logic
    return super._convertToShares(assets, rounding);
}
```

The comment directly above it cites [solmate issue 178](#) and "a known issue mentioned in OpenZeppelin docs". The intent was to fix the rounding-to-zero problem. The effect is that the fix for that problem disables the defence against this one. This is the pre-4.9 workaround layered on top of a library that has since solved it properly, and the workaround wins.

`totalAssets()` has the same shape of override at `src/PearVault.sol:458-460`, returning only the local balance when supply is zero and hiding everything held remotely from the same calculation.

Two: `_convertToAssets()` is not overridden, so the two directions disagree. Deposits at zero supply go through the raw 1:1 branch. Redemptions still use OpenZeppelin's virtual maths. That mismatch is what creates the leftover pool: burning the last share leaves assets behind that the virtual share was supposed to keep locked away forever, and the 1:1 deposit branch hands them to the next depositor.

Three: the guarantee assumes the vault empties once. OpenZeppelin's model treats zero supply as the state a vault starts in. It never expects a live vault to go back there. Nothing in this contract stops it. `redeem()` has no floor, so the owner can walk the supply back down to zero and re-enter the reset branch whenever they like. OpenZeppelin has a [tracking issue for exactly this](#), which describes the problem as "once a vault is emptied, the conversion rate of shares (or the share price) is reset to the initial value" and recommends "a requirement that, once a vault is initialized (and funded), the totalSupply must remain to be greater than a certain threshold".

That last point is why the usual fixes are not enough on their own:

- **Raising `_decimalsOffset()` does nothing here.** The override returns before `super._convertToShares()` is reached at zero supply, so a larger offset is never applied on the deposit that matters.
- **Rejecting zero share deposits is not enough either.** It stops the depositor from losing their money, but it turns the attack into a denial of service instead: the owner donates, and honest deposits revert. The measured cost of that griefing is 297,029 raw units, which is cheap.
- **Seeding the vault at deploy time is not enough** unless the seed cannot be redeemed. A seed the owner can withdraw is just a slower path back to zero supply.

The fix that actually closes it is the one OpenZeppelin's issue points at: a floor on `totalSupply()` that holds for the life of the vault.

Location of Affected Code

File: [src/PearVault.sol#L364-L370](#)

```
function loadFirstDeposit(uint256 amount) external payable nonReentrant {
    address vaultOwner_ = owner();
    if (msg.sender != vaultOwner_) revert NotAuthorized();
    if (firstDepositLoaded) revert FirstDepositAlreadyLoaded();
    if (totalSupply() != 0) revert InvalidAmount();
    // code
}
```

File: [src/PearVault.sol#L458-L460](#)

```
function totalAssets() public view override(ERC4626Upgradeable, IERC4626) returns (uint256) {
    // code
    if (totalSupply() == 0) {
        return vaultAssetBalance;
    }
    // code
}
```

File: [src/PearVault.sol#L609-L616](#)

```
function _convertToShares(uint256 assets, Math.Rounding rounding) internal view override returns (uint256) {
    if (totalSupply() == 0) {
        return assets;
    }
    // code
}
```

File: [src/PearVault.sol#L660](#)

```
function deposit(uint256 assets, address receiver) public override(ERC4626Upgradeable, IERC4626) nonReentrant
onlyActiveVault returns (uint256) {
    // code
    uint256 shares = super.deposit(assets, receiver);
    // code
}
```

Impact

A depositor can hand over the protocol minimum and receive nothing for it. Their money stays in the vault and the owner takes it out.

The attack needs no admin action, no compromised agent, no odd config and no bridge timing. It needs the owner key, roughly twice the target deposit in working capital for the length of a cycle, the setup fees, and a depositor who uses plain `deposit()` rather than `depositWithSlippage()`. Plain `deposit()` is what the docs currently point people at.

It repeats. Two cycles against one vault drain two depositors using the same donation each time, so the profit grows with the number of depositors rather than being capped by a one-time setup.

Recommendation

The one change that closes this is a permanent floor on the share supply. The rest are worth doing, but none of them is sufficient alone.

1. **Lock a seed and never let the supply fall below it.** Mint seed shares to an address that cannot transfer or redeem them, and enforce `totalSupply() >= lockedSeedShares` after setup. Make sure no withdrawal, cancellation or emergency path can burn them. This is the fix, and it is the one OpenZeppelin's own issue recommends.
2. Require the bootstrap amount to be at least `comptroller().getMinimumDeposit()`.
3. Limit the 1:1 conversion and the local only `totalAssets()` to the pre setup state. If `firstDepositLoaded` is true and the supply floor is broken, revert rather than quietly resetting the price.
4. Reject any deposit that would mint zero shares. Treat this as a backstop, not the fix. On its own, it turns the attack into griefing.
5. Point the front end, the SDK and the docs at `depositWithSlippage/assets, receiver, minSharesOut` with a non-zero `minSharesOut`. Stop documenting plain `deposit()` as the safe default.

Add regression tests for: a one raw unit seed, a zero share deposit, a full user exit with the locked seed still in place, and an owner trying to redeem down to a one share supply after setup. That last one is the case that today's code allows and that the fix has to stop.

Team Response

Fixed.

I-01

● INFO

FIXED

Two Vaults Sharing One Agent Count The Same Money Twice

Description

`totalAssets()` reads the agent's wallet balance, perp value and spot balance and treats all of it as belonging to this vault. It does not check whether another vault is reading the same agent.

The factory takes the agent address as a parameter and does not check that it is unused:

```
//src/PearVaultFactory.sol:63
```

```
function createVault(  
    address owner_,  
    address agent,  
    ...  
)
```

If two vaults are ever created with the same agent, both count the agent's balance in full. The total shares issued across the two vaults are backed by half as much money as the contracts believe.

Location of Affected Code

File: [src/PearVault.sol#L444-L469](#)

```
function totalAssets() public view override(ERC4626Upgradeable, IERC4626) returns (uint256) {  
    // code  
    uint256 agentWalletBalance = IERC20(vaultConfig.assetToken).balanceOf(agent);  
    // code  
    uint256 agentPerpValue = HyperliquidHelper.getAgentPerpValue(agent);  
    uint256 agentAssetSpot = HyperliquidHelper.getAgentSpotBalance(  
        agent,  
        vaultConfig.assetTokenId  
    ) / 100;  
    // code  
}
```

File: [src/PearVaultFactory.sol#L63-L102](#)

Impact

Both vaults overstate what they are worth, so shares in both are worth less than they claim. Whichever vault redeems first drains the shared balance and the second is left short.

This needs a provisioning mistake to happen. Vault creation is behind `onlyWhitelisted`, and the fleet we looked at had 32 different agents across 32 vaults, so it is not happening today. It is Informational because it depends on the off-chain service getting it wrong, not because the contract would cope.

Recommendation

Track used agents in the factory and reject a repeat:

```
if (agentInUse[agent]) revert AgentAlreadyAssigned();  
agentInUse[agent] = true;
```

The contract should not be relying on the off-chain service to get this right, especially since the whole trust model rests on one agent belonging to one vault.

Team Response

Fixed.

I-02

● INFO

FIXED

Nothing Checks That A Config's Asset And System Address Use The Same Unit

Description

`totalAssets()` adds together numbers from several places: an ERC-20 balance on the EVM side, a perp account value from HyperCore, a spot balance scaled from 8 decimals to 6, and a Felix vault position. The addition only makes sense if all of those are denominated in the same thing.

Nothing enforces that. `createVaultConfig()` checks that the addresses are not zero and the name is not empty. It does not check that `assetToken`, `assetTokenId`, `systemAddress` and `felixVault` describe the same asset.

Location of Affected Code

File: [src/Comptroller.sol#L497-L509](#)

```
function createVaultConfig(IVaultStructs.VaultConfig memory config) external onlyOwner returns (uint256) {
    require(config.assetToken != address(0), "Invalid asset token");
    require(config.systemAddress != address(0), "Invalid system address");
    require(bytes(config.assetName).length > 0, "Empty asset name");
    // felixVault can be address(0) if not used
    // enabled flag is set by the caller

    uint256 configId = _vaultConfigs.length;
    _vaultConfigs.push(config);

    emit VaultConfigCreated(configId, config.assetToken, config.assetName);
    return configId;
}
```

The sum that assumes they match is at `src/PearVault.sol:493-500`.

File: [src/PearVault.sol#L493-L500](#)

Impact

A config that mixes assets would give a share price that is meaningless, and every deposit and redemption after it would be mispriced.

Two things keep this informational. Only the Comptroller owner can create a config, and `initialize()` copies the config into vault storage, so changing a config later cannot corrupt vaults that already exist, only new ones.

Worth saying plainly: the only thing preventing this is that somebody types the right values. The current setup relies on USDC and USDHL trading close to par. If USDHL ever moved off par, the sum would be wrong with no config change at all.

Recommendation

Check what can be checked at config creation. Compare

`IERC20Metadata[config.assetToken].decimals()` against what the scaling in `totalAssets()` assumes, and reject a config whose decimals do not match. Check that `assetTokenId` and `systemAddress` refer to the same asset as `assetToken`.

The Felix leg is already covered. A previous review raised the unvalidated Felix asset, and the fix landed in `depositToFelixVault()`, which reverts with `FelixVaultAssetMismatch` if the Felix vault's asset does not match. That check runs at deposit time rather than config creation, which is late but sufficient, so no further action is needed on Felix.

For the depeg case, consider reading a price for each leg rather than assuming parity, or write down the parity assumption somewhere visible so whoever adds the next asset knows it exists.

Team Response

Fixed.

I-03

● INFO

FIXED

A Second Withdrawal Request Subtracts Locked Shares Twice

Description

This bug was introduced by the fix for an earlier finding, the same fix for the previous review's M-02. Locking shares stopped requesters from moving them out from under a pending request, and that was taken care of. What was not taken care of is the accounting. Commit `51b7c37` added `userLockedShares` on top of a transfer that already removes the shares from the user's balance, so the same shares are now counted twice. Neither the mapping nor the transfer existed before that commit.

`requestWithdrawal` works out what a user can still withdraw as `balanceOf(msg.sender) - userLockedShares[msg.sender]`.

Those two numbers already overlap. When the first request is made, the shares are physically moved to the vault with `_transfer(msg.sender, address(this), shares)`, so they have already left `balanceOf()`. They are then also added to `userLockedShares`.

So on a second request, the same shares are removed twice. A user who queues half their position can only queue about a quarter of what remains, and so on.

Location of Affected Code

File: [src/PearVault.sol#L721-L729](#)

```
function requestWithdrawal(uint256 shares, address receiver) external override nonReentrant validAmount(shares) {  
    // code  
    uint256 userBalance = balanceOf(msg.sender);  
    uint256 lockedShares = userLockedShares[msg.sender];  
  
    if (userBalance < lockedShares) {  
        // Edge case: shouldn't happen but handle gracefully  
        revert InsufficientBalance();  
    }  
  
    uint256 availableShares = userBalance - lockedShares;  
    // code  
}
```

File: [src/PearVault.sol#L749-L752](#)

```
function requestWithdrawal(uint256 shares, address receiver) external override nonReentrant validAmount(shares) {  
    // code  
  
    // Lock the shares by transferring them to the vault  
    // This prevents users from transferring shares after requesting withdrawal  
    _transfer(msg.sender, address(this), shares);  
  
    // Track locked shares for the user and globally  
    userLockedShares[msg.sender] += shares;  
  
    // code  
}
```

Impact

A user cannot queue as much as they should be able to in one go. It is not a loss: the first request still works, cancelling a request returns the shares, and the user can queue the rest once earlier requests settle. It is a usability bug, which is why it is rated at the QA tier.

Recommendation

Since the transfer already removes the shares from the balance, the balance on its own is the right number:

```
uint256 availableShares = balanceOf(msg.sender);
```

Keep `userLockedShares` for accounting and for the cost basis maths in `_withdrawWithFee()`, but stop subtracting it here. Check every other reader of `userLockedShares` before changing this, since the same double count may be assumed elsewhere.

Team Response

Fixed.

I-04

● INFO

FIXED

`maxDeposit()` And `maxMint()` Ignore The Per User Cap

Description

This bug was not introduced by a fix. It is an earlier finding that was only partly taken care of. The previous review reported L-09 part two, the `max*` functions not factoring in the vault's limits, and listed four limits: vault status, first deposit loaded, pending shares, and the per-user deposit cap from the Comptroller. Commit `57e9f0d` took care of the first three. The per-user cap was not, and it is still missing. Before that commit, `maxDeposit()` was not overridden at all and the inherited version ignored the cap as well, so this gap predates the fix and survived it.

`maxDeposit()` returns zero when the vault is paused or not set up yet, and otherwise hands off to the parent, which returns `type(uint256).max`.

`deposit()` then enforces a per-user cap based on the receiver's PEAR tier. So `maxDeposit()` promises an unlimited deposit and `deposit()` reverts with `DepositLimitExceeded`. ERC-4626 says `maxDeposit()` must return an amount that would actually succeed.

`maxMint()` has the same gap.

Location of Affected Code

File: [src/PearVault.sol#L902-L907](#)

```
function maxDeposit(address receiver) public view override(ERC4626Upgradeable, IERC4626) returns (uint256) {
    if (status != VaultStatus.Active || !firstDepositLoaded) {
        return 0;
    }
    return super.maxDeposit(receiver);
}
```

File: [src/PearVault.sol#L912-L917](#)

```
function maxMint(address receiver) public view override(ERC4626Upgradeable, IERC4626) returns (uint256) {
    if (status != VaultStatus.Active || !firstDepositLoaded) {
        return 0;
    }
}
```

```
return super.maxMint(receiver);  
}
```

The cap that these ignore is at

File: [src/PearVault.sol#L651-L656](#)

Impact

An integrator sizing a deposit from `maxDeposit()` gets a reverting transaction. No funds are at risk. It costs gas and it breaks the standard's promise.

Recommendation

Return the real remaining allowance:

```
function maxDeposit(address receiver) public view override returns (uint256) {  
    if (status != VaultStatus.Active || !firstDepositLoaded) return 0;  
    if (receiver == owner()) return type(uint256).max;  
  
    uint256 maxAllowed = comptroller().getMaxAllowedInvestment(receiver);  
    uint256 used = userDepositAmount[receiver];  
    return used >= maxAllowed ? 0 : maxAllowed - used;  
}
```

Mirror it in `maxMint()` by converting the result to shares.

Team Response

Fixed.

I-05

● INFO

FIXED

`_minCashFlowBPS()` **Mislabels Live Vault
Balance as Total Deposits Percentage**

Description

The storage variable's doc comment states the buffer is sized as a percentage of total deposits:

```
/// @notice Minimum cash to keep in vault as percentage of total deposits (BPS)  
uint256 private _minCashFlowBPS;
```

But `_autoRebalanceToCore()` actually computes the target off the vault's current live balance, not any tracked cumulative-deposits figure:

```
uint256 targetCash = (vaultBalance * _minCashFlowBPS) / 10000;  
vaultBalance is vaultToken().balanceOf(address(this))
```

At the moment of the call, it shrinks every time a sweep ships funds out, and the contract never tracks "total deposits ever made" anywhere.

The external doc book, written later (contract-reference.md, from the docs-rewrite commit), correctly describes the real behavior: $\text{targetCash} = \text{balance} * \text{minCashFlowBPS} / 10000$. So this is a case of internal documentation drift: the newer external doc is accurate, but the original inline comment sitting directly on the variable (the thing most developers/auditors would trust as authoritative) was never corrected to match.

Location of Affected Code

File: [src/PearVault.sol#L115-L117](#)

```
// @notice Minimum cash to keep in vault as a percentage of total deposits (BPS)  
uint256 private _minCashFlowBPS;
```

Impact

Incorrect Documentation

Recommendation

Correct the inline comment to state the true behavior.

Team Response

Fixed.



Your security partner, for the long term

[SHIELDIFY.ORG](https://shieldify.org) · [X.COM/SHIELDIFYSEC](https://x.com/shieldifysec) · [LINKEDIN](#)